

Protocole expérimental

Évaluation du fine-tuning pour le transfert d'apprentissage en régression

16 juillet 2026

Objectif

L'objectif de cette expérience est d'évaluer le **fine-tuning** comme approche de transfert d'apprentissage dans un problème de régression à partir de données simulées.

La première étape consiste à simuler deux jeux de données, un pour chaque modèle générateur. Dans chacun d'eux, les observations sont réparties entre un **domaine source** et un **domaine cible**. Plusieurs stratégies d'apprentissage sur le domaine cible seront ensuite comparées afin de mesurer l'intérêt du transfert depuis le domaine source.

Variables simulées

On considère quatre variables explicatives : deux variables catégorielles et deux variables continues.

$$X_1 \in \{a_1, b_1\}, \quad X_2 \in \{a_2, b_2, c_2\}, \quad X_3 \in \mathbb{R}, \quad X_4 \in \mathbb{R}.$$

La variable X_1 distingue les deux domaines : une modalité correspond au domaine source et l'autre au domaine cible. La variable X_2 possède trois modalités. Les variables X_3 et X_4 sont continues.

Pour les équations ci-dessous, $i \in \{1, 2\}$ désigne la modalité de X_1 , $j \in \{1, 2, 3\}$ celle de X_2 et k indexe les observations appartenant à la combinaison (i, j) .

Modèles générateurs

Deux mécanismes de génération sont envisagés. Ils permettent de faire varier la complexité de la relation entre les variables explicatives et la réponse.

Modèle 1 : effets principaux et pente propre au domaine

$$\mathcal{M}_1 : \quad Y_{ijk} = \mu + \mu_{1,i} + \mu_{2,j} + (\beta_3 + \gamma_{1,i})X_{3,ijk} + \varepsilon_{ijk}.$$

Ce premier modèle comprend les effets principaux de X_1 et X_2 , ainsi qu'une interaction entre X_1 et X_3 . La pente associée à X_3 peut donc différer entre les domaines source et cible.

Modèle 2 : interactions d'ordre supérieur

$$\mathcal{M}_2 : \quad Y_{ijk} = \mu + \mu_{1,i} + \mu_{12,ij} + (\beta_3 + \gamma_{12,ij})X_{3,ijk} + (\beta_{34} + \gamma_{2,j})X_{3,ijk}X_{4,ijk} + \varepsilon_{ijk}.$$

Le second modèle introduit un effet propre à chaque combinaison des modalités de X_1 et X_2 , une pente de X_3 propre à chacune de ces combinaisons, ainsi qu'une interaction entre X_2 , X_3 et X_4 . Il représente ainsi un mécanisme de génération plus complexe.

Définition des paramètres

- μ est l'ordonnée à l'origine globale ;
- $\mu_{1,i}$ est l'effet associé à la modalité i de X_1 ;
- $\mu_{2,j}$ est l'effet associé à la modalité j de X_2 ;
- $\mu_{12,ij}$ est l'effet associé à la combinaison des modalités i de X_1 et j de X_2 ;
- β_3 est la pente moyenne associée à X_3 ;
- $\gamma_{1,i}$ décrit la modification de la pente de X_3 selon X_1 ;
- $\gamma_{12,ij}$ décrit la modification de la pente de X_3 selon la combinaison (X_1, X_2) ;
- β_{34} est le coefficient moyen de l'interaction entre X_3 et X_4 ;
- $\gamma_{2,j}$ décrit la modification de cette interaction selon X_2 ;
- ε_{ijk} est le terme d'erreur, supposé indépendant et identiquement distribué selon

$$\varepsilon_{ijk} \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2).$$

Proposition de valeurs des paramètres

On associe a_1 au domaine source et b_1 au domaine cible. Les valeurs ci-dessous constituent un premier scénario de simulation : elles créent une différence modérée entre les deux domaines, tout en conservant un rapport signal sur bruit suffisant pour faire apparaître la structure des modèles.

Paramètres du modèle 1

On propose :

$$\mu = 2, \quad \beta_3 = 1,5, \quad \sigma = 1,$$

X_1	a_1	b_1	X_2	a_2	b_2	c_2
$\mu_{1,i}$	0	1	$\mu_{2,j}$	0	-0,75	0,75
$\gamma_{1,i}$	0	-0,5				

Dans ce scénario, le domaine cible présente une réponse moyenne plus élevée d'une unité. La pente de X_3 vaut 1,5 dans le domaine source et 1 dans le domaine cible.

Paramètres du modèle 2

On propose :

$$\mu = 2, \quad \beta_3 = 1,2, \quad \beta_{34} = 0,8, \quad \sigma = 1,$$

$$\frac{X_1}{\mu_{1,i}} \left| \begin{array}{cc} a_1 & b_1 \\ 0 & 0,75 \end{array} \right.,$$

$$\begin{array}{c|ccc} \mu_{12,ij} & a_2 & b_2 & c_2 \\ \hline a_1 \text{ (source)} & 0 & -1 & 1 \\ b_1 \text{ (cible)} & 0 & -0,5 & 1,5 \end{array}, \quad \begin{array}{c|ccc} \gamma_{12,ij} & a_2 & b_2 & c_2 \\ \hline a_1 \text{ (source)} & 0 & 0,4 & -0,4 \\ b_1 \text{ (cible)} & -0,3 & 0,7 & -0,8 \end{array},$$

$$\frac{X_2}{\gamma_{2,j}} \left| \begin{array}{ccc} a_2 & b_2 & c_2 \\ 0 & 0,4 & -0,3 \end{array} \right. .$$

Ces valeurs induisent des différences entre les domaines qui dépendent de la modalité de X_2 . La pente de X_3 varie selon chaque combinaison de X_1 et X_2 , tandis que l'effet de l'interaction X_3X_4 varie selon X_2 . Le second scénario est ainsi plus complexe que le premier, tout en gardant des coefficients d'amplitude comparable.

Protocole de simulation des données

Pour chacun des deux modèles, la simulation suivra les étapes suivantes :

1. Construire un tableau de $n = 2000$ observations contenant X_1 et X_2 . Les observations seront réparties de manière aussi équilibrée que possible entre les six combinaisons de modalités de (X_1, X_2) , soit 333 ou 334 observations par combinaison.
2. Ajouter les variables continues X_3 et X_4 , générées indépendamment selon des lois normales centrées réduites :

$$X_3 \sim \mathcal{N}(0, 1), \quad X_4 \sim \mathcal{N}(0, 1).$$

La variable X_4 est conservée dans les deux jeux de données, bien qu'elle n'intervienne que dans le modèle 2. Les variables X_3 et X_4 sont également supposées indépendantes de X_1 , de X_2 et du terme d'erreur ε .

3. Calculer la réponse Y à partir de l'équation du modèle considéré, en utilisant les paramètres proposés dans la section précédente, puis ajouter le terme d'erreur ε .

Cette procédure produira deux jeux de données simulées, un pour chaque modèle générateur.

Visualisation des données simulées

Pour chacun des deux jeux de données, représenter la distribution de la réponse Y en fonction de X_1 . Cette visualisation permettra de comparer les distributions obtenues dans les domaines source et cible.

Environnement de mise en œuvre

Le protocole sera implémenté en **Python**. La bibliothèque **PyTorch** sera utilisée pour définir les embeddings et les MLP, entraîner les modèles, appliquer l'arrêt anticipé et produire les prédictions. Les bibliothèques NumPy et pandas pourront être utilisées pour la manipulation des données, tandis que Matplotlib et Seaborn serviront à réaliser les visualisations.

Afin de garantir la reproductibilité, les versions de Python et des bibliothèques seront enregistrées. Les graines aléatoires de Python, NumPy et PyTorch seront fixées et sauvegardées pour les découpages initiaux ainsi que pour chacune des dix répétitions. Si l'expérience est exécutée sur GPU, les options déterministes de PyTorch seront activées dans la mesure du possible.

Expérience de transfert d'apprentissage par fine-tuning

L'expérience est conduite séparément sur chacun des deux jeux de données simulées. Conformément aux conventions précédentes, a_1 désigne le domaine source et b_1 le domaine cible.

Les quatre variables X_1 , X_2 , X_3 et X_4 sont utilisées comme entrées des réseaux, après un prétraitement identique pour toutes les méthodes. Les variables catégorielles X_1 et X_2 sont représentées par des embeddings appris conjointement avec le réseau. Les modalités sont encodées par des indices entiers avant leur passage dans les couches d'embedding.

Pour chaque jeu de données, les découpages initiaux sont réalisés une seule fois :

1. Réserver aléatoirement la moitié des 1 000 observations du domaine cible ($X_1 = b_1$) pour constituer un jeu de test cible fixe. Les 500 observations restantes forment le réservoir d'apprentissage cible.
2. Séparer une seule fois les 1 000 observations du domaine source ($X_1 = a_1$) en un jeu d'entraînement source fixe (80 %, soit 800 observations) et un jeu de validation source fixe (20 %, soit 200 observations).
3. Pour chaque répétition $r \in \{1, \dots, 10\}$:
 - a. entraîner, depuis une nouvelle initialisation aléatoire, un MLP sur le jeu d'entraînement source fixe, en utilisant le jeu de validation source fixe pour l'arrêt anticipé. Le réseau obtenu constitue le modèle préentraîné de la répétition r ;
 - b. utiliser les 500 observations du réservoir d'apprentissage cible pour construire des échantillons emboîtés

$$S_{10}^{(r)} \subset S_{50}^{(r)} \subset S_{100}^{(r)} \subset S_{200}^{(r)} \subset S_{500}^{(r)}.$$

La répartition des modalités de X_2 est rendue aussi équilibrée que possible dans chaque échantillon ;

- c. pour chaque taille $n \in \{10, 50, 100, 200, 500\}$, réserver aléatoirement 20 % de $S_n^{(r)}$ pour la validation cible. Les effectifs d'entraînement et de validation sont respectivement (8, 2), (40, 10), (80, 20), (160, 40) et (400, 100). Les mêmes sous-ensembles sont utilisés par les trois stratégies ;
- d. affiner une copie du modèle préentraîné de la répétition r (*fine-tuning*) à partir du sous-ensemble d'entraînement cible, avec un arrêt anticipé fondé sur la validation cible ;
- e. entraîner depuis une nouvelle initialisation aléatoire (*from scratch*) un deuxième MLP de même architecture sur les seuls sous-ensembles d'entraînement et de validation cibles ;
- f. entraîner depuis une nouvelle initialisation aléatoire un troisième MLP sur l'ensemble combinant les 1 000 observations source et le sous-ensemble d'entraînement cible. Le sous-ensemble de validation cible guide l'arrêt anticipé. Ce réseau est appelé **modèle combiné** ;
- g. évaluer les trois modèles sur le même jeu de test cible fixe et calculer leur racine de l'erreur quadratique moyenne (RMSE) :

$$\text{RMSE} = \sqrt{\frac{1}{N_{\text{test}}} \sum_{q=1}^{N_{\text{test}}} (Y_q - \hat{Y}_q)^2}.$$

- h. appliquer également la fonction génératrice exacte f aux variables explicatives de ce même jeu de test, sans ajouter un nouveau bruit, et calculer

$$\text{RMSE}_{\text{oracle}} = \sqrt{\frac{1}{N_{\text{test}}} \sum_{q=1}^{N_{\text{test}}} (Y_q - f(X_q))^2}.$$

Cette valeur est calculée une seule fois par modèle générateur, puisque le jeu de test est fixe et commun à toutes les répétitions.

4. Pour chaque modèle générateur, chaque stratégie et chaque valeur de n , représenter par un boxplot les dix RMSE obtenues. Les observations individuelles seront superposées aux boxplots afin de rendre visibles les dix répétitions. Les trois stratégies comparées sont le fine-tuning, l'entraînement cible *from scratch* et l'apprentissage combiné source-cible. Une ligne horizontale noire pointillée indique, sur chaque panneau, la $\text{RMSE}_{\text{oracle}}$ du modèle générateur correspondant.

Plancher de RMSE lié au bruit

Comme le terme d'erreur suit

$$\varepsilon \sim \mathcal{N}(0, \sigma^2)$$

et que la réponse Y n'est pas standardisée, la RMSE irréductible dans la population est directement exprimée dans l'unité de Y et vaut

$$\text{RMSE}_{\text{irréductible}} = \sqrt{\mathbb{E}(\varepsilon^2)} = \sigma.$$

Dans le scénario retenu, $\sigma = 1$: le plancher théorique de RMSE vaut donc 1. Pour un prédicteur $\hat{f}(X)$ de la fonction génératrice $f(X)$, l'erreur peut être décomposée, en population, sous la forme

$$\mathbb{E}[(Y - \hat{f}(X))^2] = \sigma^2 + \mathbb{E}[(f(X) - \hat{f}(X))^2],$$

sous l'hypothèse que le bruit est centré et indépendant des variables explicatives. Le premier terme correspond au bruit irréductible ; le second regroupe les erreurs d'approximation et d'estimation du modèle.

Sur un jeu de test fini, même le modèle oracle, qui connaît exactement f , ne présente pas nécessairement une RMSE égale à 1, car la variance empirique du bruit fluctue d'un échantillon à l'autre. Pour la graine de simulation et les jeux de test fixes de 500 observations utilisés dans cette expérience, la RMSE oracle propre à chaque modèle sera calculée à partir des réalisations du bruit sur son jeu de test et reportée avec les résultats.

Ces valeurs constituent les références empiriques les plus pertinentes pour interpréter les résultats des MLP. Lorsque le nombre d'observations cibles augmente, un MLP suffisamment flexible et correctement entraîné devrait s'en rapprocher. Il peut néanmoins rester au-dessus à cause de la taille finie des échantillons d'entraînement et de validation, de l'optimisation, de l'arrêt anticipé, du *dropout*, de la régularisation et de sa capacité d'approximation. En outre, augmenter seulement le nombre d'observations sources ne garantit pas d'atteindre ce plancher dans le domaine cible lorsque les relations diffèrent entre les deux domaines.

Arrêt anticipé

Pour chaque entraînement, la fonction de perte est évaluée sur le jeu de validation à la fin de chaque époque. L'entraînement est interrompu si cette perte ne s'améliore pas pendant 20 époques consécutives (*patience* de 20), avec un maximum fixé à 500 époques. Les poids correspondant à la meilleure perte de validation sont ensuite restaurés avant l'évaluation sur le jeu de test.

Le jeu de validation cible est strictement identique pour les trois stratégies au sein d'une répétition. Avec seulement deux observations de validation lorsque $n = 10$, le critère d'arrêt est nécessairement instable ; les répétitions permettront de mesurer la variabilité qui en résulte.

Architecture du MLP et paramètres d'entraînement

Une architecture compacte est retenue afin de limiter le surapprentissage, en particulier lorsque peu d'observations cibles sont disponibles. Une dimension d'embedding de 1 est utilisée pour X_1 , qui possède deux modalités, et une dimension de 2 pour X_2 , qui en possède trois. Les deux embeddings sont concaténés avec les variables continues X_3 et X_4 . La partie dense du réseau reçoit donc cinq valeurs : $1 + 2 + 1 + 1 = 5$.

Élément	Valeur proposée
Embedding de X_1	2 modalités $\rightarrow$ dimension 1
Embedding de X_2	3 modalités $\rightarrow$ dimension 2
Architecture dense après concaténation	5 entrées $\rightarrow$ 32 neurones $\rightarrow$ 16 neurones $\rightarrow$ 1 sortie
Activation des couches cachées	ReLU
Activation de sortie	aucune (sortie linéaire)
Fonction de perte	erreur quadratique moyenne (MSE)
Optimiseur	Adam
Taux d'apprentissage — préentraînement, <i>from scratch</i> et modèle combiné	10^{-3}
Taux d'apprentissage — fine-tuning	10^{-4}
Taille des lots	$\min(32, N_{\text{train}})$
Pénalisation L_2 sur les poids	10^{-4}
Dropout	0,1 après chaque couche cachée
Nombre maximal d'époques	500
Arrêt anticipé	patience de 20 époques, seuil minimal d'amélioration de 10^{-4}
Initialisation	initialisation de He pour les couches denses ; petite initialisation aléatoire pour l'embedding de X_2

Le taux d'apprentissage plus faible lors du fine-tuning vise à adapter progressivement les poids préentraînés sans dégrader brutalement les représentations apprises sur le domaine source. Toutes les couches sont néanmoins laissées entraînaibles, car le réseau est petit et les différences entre les domaines peuvent concerner à la fois les effets principaux et les interactions.

Pendant le préentraînement source, seule la ligne de l'embedding de X_1 correspondant à a_1 reçoit des gradients ; celle de b_1 n'est pas observée. Afin que la représentation initiale du domaine cible ne soit pas purement aléatoire, les deux lignes de cet embedding sont initialisées à zéro. La ligne associée à a_1 est apprise pendant le préentraînement, tandis que celle associée à b_1 peut être apprise pendant le fine-tuning ou l'apprentissage combiné. La même règle d'initialisation

est appliquée aux trois stratégies. Tous les embeddings et toutes les couches denses restent entraînaibles pendant le fine-tuning.

Les variables continues X_3 et X_4 sont standardisées à partir des moyennes et écarts-types du jeu d'entraînement source. Cette transformation est ensuite conservée sans modification pour le fine-tuning, l'apprentissage cible, le modèle combiné, la validation et le test. La réponse Y n'est pas standardisée. Ces choix constituent une configuration de référence fixée avant l'analyse ; une étude ultérieure pourra évaluer la sensibilité aux dimensions du réseau, au taux d'apprentissage et à la régularisation.

Rôle de la variable de domaine X_1

Dans le modèle préentraîné sur le domaine source comme dans le modèle entraîné uniquement sur les n observations cibles, une seule modalité de X_1 est observée. L'embedding de la modalité absente n'est donc pas appris, et l'effet de l'embedding observé ne peut pas être distingué d'un décalage du biais de la première couche dense.

En revanche, le modèle combiné observe les deux modalités de X_1 . Il peut ainsi apprendre directement un décalage entre les domaines et, grâce aux couches non linéaires du MLP, des relations qui dépendent du domaine. L'inclusion de X_1 est donc surtout informative pour cette troisième stratégie.

Afin de mesurer l'influence de ce choix, une analyse de sensibilité pourra répéter l'expérience en excluant X_1 des entrées. La comparaison principale conservera néanmoins X_1 dans les trois modèles afin que ceux-ci disposent exactement des mêmes variables explicatives.

Le jeu de test cible ne doit être utilisé ni pour entraîner les modèles ni pour choisir l'architecture ou les hyperparamètres. Ces choix doivent être fixés au préalable ou effectués à partir des seules données d'apprentissage et de validation.